

Local-First Short-Form Video Production in 2026: A Best-Practice Pipeline for RTX 4090 Workstations

Executive summary

Your proposed architecture is **very close to a 2026 best-practice workflow** for producing faceless, AI-assisted TikTok/Reels/YouTube Shorts on a powerful single-GPU workstation:

```
script
  ↓
local TTS
  ↓
approved narration master
  ↓
word-level alignment
  ↓
captions + proposed visual beats
  ↓
human-approved storyboard/timeline
  ↓
still generation
  ↓
selective image-to-video generation
  ↓
media normalization
  ↓
Windows / CapCut
  ↓
human editorial finishing
```

The architecture is sound because it assigns each problem to the right class of tool. The narration becomes the master clock; alignment solves timing deterministically; an LLM proposes semantic visuals without controlling final timings; image generation establishes composition cheaply; video generation is reserved for shots where actual motion adds value; and CapCut remains a human-controlled finishing environment. WhisperX itself is built around forced alignment for word-level timing, ComfyUI now exposes first-class workflow automation, and CapCut Desktop can import SRT files into synchronized editable caption blocks.

1

I would, however, make several material changes to the stack you originally outlined:

Original idea	Recommended 2026 version	Why
Chatterbox generally	Chatterbox-Turbo for English ; Multilingual V3 when needed	Turbo is the purpose-built fast English option; V3 is now the current multilingual general model. ²
WhisperX → timestamps	WhisperX, pinned and validated	Excellent architecture, but recent issue reports make version pinning and alignment QA important.
<code>suggested_shots.csv</code>	Machine proposal only	Human-approved <code>timeline.csv</code> should become authoritative.
FLUX.1 Schnell	FLUX.2 [klein] 4B	Newer, Apache 2.0, ~8 GB VRAM, four-step production inference, multi-reference editing. ³
Wan + Hunyuan equally	Wan 2.2 as primary; Hunyuan optional	Wan T12V-5B explicitly targets 24 GB/RTX 4090 and is Apache 2.0. Hunyuan has a significant territorial license problem for globally published content. ⁴
Generate many T2V clips	Still-first → I2V-first	Much greater composition/identity control and fewer wasted expensive generations.
Manually use ComfyUI	Save API workflow JSON and automate queues	ComfyUI officially exposes HTTP/WebSocket APIs and CLI workflow execution. ⁵
Normalize everything to 30 fps	Normalize codec/ resolution; preserve native frame rate where practical	Wan natively generates 24 fps. Arbitrarily changing 24→30 merely duplicates/ interpolates frames unless you use actual motion interpolation. ⁶
CapCut as another AI stage	CapCut as finishing/ editorial stage	Import already-timed SRT and assets, then use CapCut for typography, pacing, sound, transitions, framing and final output. ⁷

My overall assessment is **about 9/10 as a production architecture** after those changes. It is particularly strong for a solo creator because it minimizes recurring API cost without attempting the counterproductive goal of automating taste.

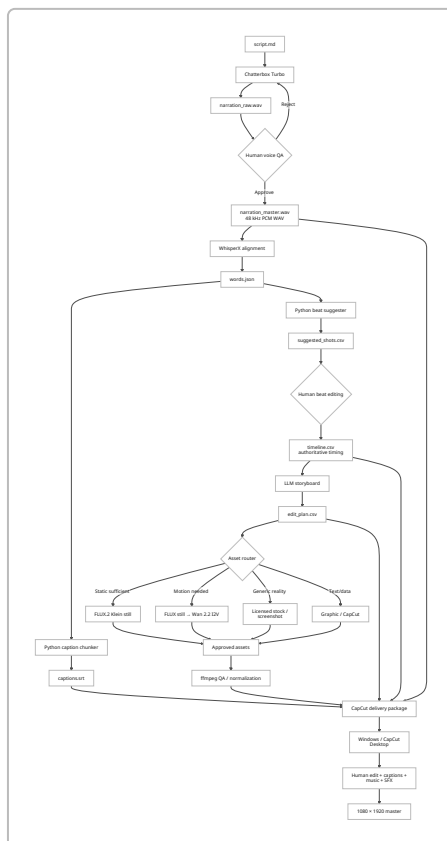
The most consequential recommendation is legal rather than technical: **for content intended to be publicly available worldwide, I would default to Wan 2.2 and avoid HunyuanVideo-1.5 unless you have obtained legal advice or can enforce territorial distribution restrictions.** Tencent's current HunyuanVideo-1.5 license defines its permitted Territory as the world excluding the EU, UK and South Korea and says the Hunyuan Works *and outputs/results* may not be used, reproduced, modified, distributed or displayed outside that Territory. A globally available YouTube/TikTok/Reels upload therefore creates an obvious license-compliance concern. ⁸

Architecture and production standards

The central principle should be:

Audio establishes time; humans establish editorial beats; AI generates assets; the NLE establishes the final experience.

Do not let the LLM, video model or CapCut independently invent competing timelines.



This design also fits current platform guidance. TikTok's own creative guidance recommends vertical 9:16, sound/music, at least 720p, and keeping important content within interface-safe areas; it emphasizes getting the proposition into the first three seconds and giving special attention to the first six seconds. Those recommendations are written for advertising, so they are useful directional evidence rather than immutable rules for organic videos. ⁹ TikTok's June 2026 technical ad specification likewise recommends 9:16 vertical media and explicitly notes that safe-zone dimensions depend on caption length and interface elements. ¹⁰ YouTube currently classifies square or vertical uploads of up to three minutes as Shorts for standard channels when they meet its date rules. ¹¹

Recommended project master standard

Item	Production recommendation
Canvas	1080 × 1920, 9:16
Color	SDR / Rec.709 unless you deliberately establish an HDR workflow
Still master	PNG, sRGB, ideally $\geq 1080 \times 1920$ or model-native portrait followed by crop/upscale
Wan generation	704×1280, 24 fps for its 720p-class portrait TI2V-5B path
Narration master	WAV, PCM, mono, 48 kHz, 24-bit
SFX/music source	WAV 48 kHz when available; high-quality compressed source otherwise
Intermediate video	MP4/H.264/yuv420p or a higher-quality editing codec if storage permits
Source frame rate	Preserve native 24/25/30 fps unless there is a reason to convert
Final social master	1080×1920 , H.264, AAC, 24 or 30 fps depending project cadence
Captions	UTF-8 <code>.srt</code> , generally 2–5 words per caption event
Timing precision	Milliseconds internally
File naming	ASCII-safe, zero-padded IDs: <code>shot_001_take_03.mp4</code>

Wan's official TI2V-5B supports both T2V and I2V at 720p/24 fps; its reference repository gives `1280*704` and `704*1280` as the two 720p orientations and explicitly says the 24 GB configuration runs on an RTX 4090 with CPU/model offloading flags. ¹²

A good project directory is:

```

video_0042/
├── 00_admin/
│   ├── project.yaml
│   ├── build_log.csv
│   ├── licenses.csv
│   ├── checksums.sha256
│   └── environment/
│       ├── pip-freeze.txt
│       ├── comfy_commit.txt
│       ├── custom_nodes.csv
│       └── models.csv
├── 01_script/
│   ├── script_v01.md
│   ├── script_locked.md
│   └── pronunciation.yaml

```

```
|
|
|— 02_voice/
|   |— voice_reference.wav
|   |— narration_raw/
|   |   └─ narration_master.wav
|
|— 03_alignment/
|   |— whisperx_raw.json
|   |— words.json
|   |— captions.srt
|   |   └─ alignment_review.csv
|
|— 04_timeline/
|   |— suggested_shots.csv
|   |— timeline.csv
|   |   └─ edit_plan.csv
|
|— 05_stills/
|   |— candidates/
|   |   └─ approved/
|
|— 06_video/
|   |— candidates/
|   |— approved/
|   |   └─ normalized/
|
|— 07_audio/
|   |— music/
|   |   └─ sfx/
|
|— 08_workflows/
|   |— flux2_klein_api.json
|   |— wan22_i2v_api.json
|   |   └─ workflow_manifest.json
|
|— 09_capcut_delivery/
|   |— narration_master.wav
|   |— captions.srt
|   |— timeline.csv
|   |— edit_plan.csv
|   |— video/
|   |— images/
|   |— audio/
|   |   └─ provenance/
|
|— 10_exports/
```

The extra `00_admin` data is not bureaucracy for its own sake. Generative workflows are unusually sensitive to model revisions, custom-node revisions, prompt-rewriting behavior, random seeds and dependency changes. ComfyUI itself distinguishes stable and latest/nightly releases, and its documentation notes that template availability can depend on how current the installation is. ¹³

Stage-by-stage implementation

The table below is the operational recommendation before the detailed commands.

Stage	Local default	4090 recommendation	Cloud fallback	Primary tradeoff
TTS	Chatterbox-Turbo	CUDA; medium-length narration chunks	ElevenLabs TTS	control/cost vs hosted consistency
Alignment	WhisperX	GPU, pinned stable version	Deepgram	local reproducibility vs managed reliability
Captions/ beats	Python	CPU; deterministic	CI/any Python runner	almost no reason to cloud-host
Storyboard	LLM with strict schema	Run serially, not alongside video gen	hosted LLM	local privacy vs better frontier reasoning
Stills	FLUX.2 Klein 4B	4-step distilled	BFL API	very fast local vs premium cloud models
Video	Wan 2.2 TI2V-5B	704×1280, offloading	Runway Gen-4.5 / Comfy Cloud	minutes locally vs paid burst quality/speed
Alternate video	HunyuanVideo-1.5	480p distilled for previews	hosted Hunyuan provider	speed, but problematic distribution license
Normalization	ffmpeg/ffprobe	CPU/NVENC as desired	unnecessary	deterministic local task
Finish	CapCut Desktop Windows	SRT + organized media package	CapCut Web	desktop control vs browser convenience

Voice generation. For English short-form narration, I would start with `ChatterboxTurboTTS`, not the older generic English checkpoint. The official Chatterbox repository currently presents Multilingual V3 as its latest multilingual general-purpose model and shows Turbo as a separate CUDA-capable model; it also

documents a roughly ten-second reference clip in its Turbo example. Every Chatterbox-generated audio file contains Resemble's PerTh watermark. ¹⁴

Keep the TTS environment separate from ComfyUI:

```
python3.11 -m venv .venv-chatterbox
source .venv-chatterbox/bin/activate
python -m pip install --upgrade pip
pip install chatterbox-tts
```

A minimal English generation script:

```
from pathlib import Path

import torchaudio as ta
from chatterbox.tts_turbo import ChatterboxTurboTTS

text = Path("01_script/script_locked.md").read_text().strip()

model = ChatterboxTurboTTS.from_pretrained(device="cuda")

wav = model.generate(
    text,
    audio_prompt_path="02_voice/voice_reference.wav",
)

ta.save("02_voice/narration_raw.wav", wav, model.sr)
```

That import/class pattern follows Resemble's current Turbo example. ¹⁵

For production, I recommend rendering narration in **semantic chunks rather than one 45–90 second generation**: typically paragraph/sentence groups rather than single tiny phrases. This makes pronunciation or performance fixes local rather than requiring a complete rerender. That chunk size is an editorial recommendation, not a model requirement.

Once every piece has been approved, concatenate and **resample before alignment**:

```
ffmpeg -y \
-i 02_voice/narration_assembled.wav \
-ar 48000 \
-ac 1 \
-c:a pcm_s24le \
02_voice/narration_master.wav
```

Do not change timing, silence or playback speed after alignment unless you rerun alignment.

A hosted fallback is ElevenLabs, whose current API supports production TTS and WAV downloads; use it when a local voice cannot hit the performance you need rather than simply because it is easier. ¹⁶

Word alignment. WhisperX remains an excellent fit because this is exactly the problem it was designed to solve: Whisper transcription followed by phoneme-based forced alignment to obtain finer timestamps. Its current project documentation says WhisperX uses a faster-whisper backend, VAD and wav2vec2-style alignment, and recommends its stable PyPI release rather than development code for production. ¹⁷

```
python3.11 -m venv .venv-whisperx
source .venv-whisperx/bin/activate
pip install --upgrade pip
pip install whisperx
```

A starting configuration for your 4090:

```
whisperx 02_voice/narration_master.wav \
  --model large-v2 \
  --language en \
  --device cuda \
  --compute_type float16 \
  --batch_size 8 \
  --output_dir 03_alignment
```

The WhisperX documentation explicitly uses `large-v2` plus a larger alignment model as an example when optimizing timestamp accuracy, while noting that larger alignment models were not found to help as dramatically as larger ASR models. ¹⁸

Because your narration is synthetic and you already possess the exact script, transcription accuracy is not actually the core problem: **alignment accuracy is**. I would therefore post-process the recognized tokens against `script_locked.md` and flag insertions/deletions rather than silently accepting Whisper's wording. For scripted TTS, a mismatch such as:

```
script:      "$3.4 billion"
recognized:  "three point four billion dollars"
```

should not be treated as a failed narration; it is a text-normalization/alignment issue.

WhisperX has also had recent user reports involving incorrect or missing word timestamps in some releases, so a production system should pin a tested version and maintain a tiny regression suite containing numbers, dates, acronyms and punctuation-heavy narration rather than blindly upgrading every environment. ¹⁹

A strong fallback is Deepgram: its current speech services return per-word timing data, and its 2026 Flux update explicitly added `start` and `end` timestamps to each word. ²⁰

Captions and shot suggestions. This should be **your own deterministic code**, not an AI model.

Use the aligned words as input:

```
[
  {"word": "Nobody", "start": 0.042, "end": 0.361},
  {"word": "expected", "start": 0.361, "end": 0.790},
  {"word": "this", "start": 0.810, "end": 1.002},
  {"word": "to", "start": 1.003, "end": 1.115},
  {"word": "work.", "start": 1.116, "end": 1.604}
]
```

My default caption chunking rules would be:

Rule	Default
Minimum words	2
Target	3–4
Maximum	5
Preferred maximum display	~1.3–1.6 s
Break priority	punctuation → semantic phrase → duration → word count
One-word caption	only deliberate impact word
Start	first contained word start
End	last contained word end + small visual tail if space permits
Overlap	never
Reading order	do not split determiner from noun or preposition from its object when avoidable

These are editorial heuristics, not TikTok/Meta/YouTube format requirements. They are designed to keep short-form captions visually digestible while honoring actual speech timing. Platform guidance does, however, strongly support making important content legible inside UI-safe areas rather than near the edges. ²¹

A small chunker can look like:

```

MAX_WORDS = 5
MAX_DURATION = 1.5

def should_close(chunk):
    if not chunk:
        return False

    duration = chunk[-1]["end"] - chunk[0]["start"]
    punctuation = chunk[-1]["word"].rstrip().endswith((".", "?", "!", ",", ";",
    ":"))

    return (
        len(chunk) >= MAX_WORDS
        or duration >= MAX_DURATION
        or (len(chunk) >= 2 and punctuation)
    )

```

Then produce UTF-8 SRT:

```

1
00:00:00,042 --> 00:00:00,790
Nobody expected

2
00:00:00,810 --> 00:00:01,604
this to work.

```

The **shot suggestion algorithm should be separate from the caption algorithm**. Captions answer “what words are visible together?” A shot beat answers “when does the visual idea change?” They frequently correlate, but they are not the same decision.

For automatic suggestions, target perhaps ~1.5–3.5 seconds but let semantic boundaries dominate. For a strong opening, a 0.7-second first beat can be perfectly appropriate. TikTok itself tells advertisers to establish the proposition within roughly the first three seconds and prioritize the opening six seconds; again, treat that as useful platform evidence rather than a universal organic-content formula. ⁹

Crucially:

```

suggested_shots.csv
    ↓
  HUMAN EDIT
    ↓
timeline.csv ← authoritative

```

Do **not** feed `suggested_shots.csv` directly into expensive video generation.

Storyboard generation. The LLM gets fixed timing, not permission to reinvent it. Use a structured contract such as:

```
{
  "shot_id": "shot_007",
  "start": 18.420,
  "end": 21.710,
  "duration": 3.290,
  "narration": "Nobody realized what they had discovered.",
  "visual_function": "reveal",
  "asset_type": "ai_i2v",
  "visual_description": "macro view of the unexpected material",
  "image_prompt": "...",
  "motion_prompt": "...",
  "framing": "macro, vertical, center-weighted",
  "onscreen_text": "",
  "editor_note": "cut on discovery; slow push only"
}
```

The strongest rule in your prompt should be:

```
Never alter shot_id, start, end, duration, or narration.
Return exactly one record for every supplied shot.
```

For video prompts, separate **appearance** from **motion**. Runway's current Gen-4.5 I2V guidance independently makes the same distinction: when an image already establishes appearance, the video prompt should concentrate on what moves. ²² Hunyuan's own repository likewise emphasizes detailed video prompts and offers prompt-rewriting logic, including vLLM-compatible models or Gemini as rewrite options. ²³

I recommend disabling invisible/automatic prompt rewriting for final reproducible renders unless the rewritten prompt is stored alongside the generation. A seed is not enough for reproducibility if a separate LLM silently rewrites the prompt first.

Still-image generation. In August 2026, I would make **FLUX.2 [klein] 4B**, rather than FLUX.1 Schnell, the local baseline. Black Forest Labs released it in January 2026; the official repository says the 4B model uses Apache 2.0, fits in about 8 GB VRAM, supports text-to-image plus single- and multi-reference editing, and recommends the distilled four-step variant for production use. ³

This is especially useful for a video pipeline because multi-reference editing gives you a better route to persistent characters, environments, objects and graphic styles than independently prompting each shot.

For example:

```
REFERENCE 1: approved recurring character
REFERENCE 2: approved 1920s laboratory environment
```

```
PROMPT:
Same engineer and laboratory.
Medium close-up.
He examines a small metallic sample beneath a desk lamp.
Historically plausible clothing and equipment.
Vertical framing, subject centered in safe area.
No typography.
```

On your 4090, the 4B model leaves substantial VRAM headroom compared with the 24 GB available on the card. ²⁴ FLUX.2 [dev] is a different proposition: it is 32B, is covered by BFL's non-commercial license, and the official repo describes native inference as H100-class while pointing to quantized 4090 paths. That makes it a poor default for a monetized local Shorts pipeline compared with Klein 4B. ²⁵

For a cloud emergency, BFL operates its own API; the current billing model uses credits with one credit defined as \$0.01. ²⁶

Video generation. This is where your RTX 4090 matters most, and where restraint gives the greatest efficiency gain.

Default route:

```
storyboard idea
  ↓
FLUX approved still
  ↓
Wan 2.2 TI2V-5B
  ↓
3-5 sec source clip
  ↓
human selects best take
```

Wan's official repository says TI2V-5B supports 720p/24 fps T2V and I2V, specifically targets consumer hardware such as the RTX 4090, and can produce a five-second 720p clip in under nine minutes on a consumer GPU without special optimization. ²⁷ Its reference 24 GB command is:

```
python generate.py \
  --task ti2v-5B \
  --size '704*1280' \
  --ckpt_dir ./Wan2.2-TI2V-5B \
```

```
--offload_model True \  
--convert_model_dtype \  
--t5_cpu \  
--image 05_stills/approved/shot_007.png \  
--prompt "$MOTION_PROMPT"
```

The official repository documents those offloading flags and explicitly identifies 24 GB / RTX 4090 as sufficient for this configuration. ¹²

ComfyUI is a better day-to-day frontend than repeatedly invoking the raw Wan CLI. Its official Wan 2.2 template currently packages the 5B model as:

```
models/diffusion_models/  
    wan2.2_ti2v_5B_fp16.safetensors  
  
models/text_encoders/  
    umt5_xx1_fp8_e4m3fn_scaled.safetensors  
  
models/vae/  
    wan2.2_vae.safetensors
```

and ComfyUI says its native offloading can fit the 5B workflow in substantially less VRAM than your 4090 provides. ²⁸

For shots needing intentional camera movement, explore the official ComfyUI Wan Fun Camera workflow rather than trying to coax camera motion from prose. It exposes pan/zoom-style camera controls. Its 4-step acceleration path substantially cuts generation time in ComfyUI's published 4090D tests, although ComfyUI warns that the accelerated LoRA can reduce motion dynamism. ²⁹

That gives you a useful two-tier system:

Render mode	Recommended purpose	Quality/speed strategy
Draft	decide whether motion works	low/medium resolution, accelerated steps/LoRA, 1–2 candidates
Final	approved hero shot	704×1280 Wan TI2V-5B, normal quality path, multiple seeds if needed
Static	narration merely needs visual support	skip video generation entirely; animate PNG in CapCut
Cloud hero	local model repeatedly fails	Runway Gen-4.5 or another premium provider

Current Runway Gen-4.5 supports both T2V and I2V through its API, with two-to-ten-second durations; its developer portal currently lists Gen-4.5 at \$0.12 per generated second. ³⁰ That makes it well suited to the “one impossible hero shot” fallback rather than the default pipeline.

HunyuanVideo-1.5 is technically compelling on your hardware. Its official requirements specify only 14 GB GPU memory with offloading, Linux and Python 3.10+, and Tencent's 480p step-distilled I2V model reduces 50 steps to a recommended 8 or 12; Tencent reports up to a 75% end-to-end speed reduction on a 4090 and generation within 75 seconds in that configuration. ³¹ Its OOM advice is also directly useful:

```
export PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True,max_split_size_mb:128
```

Tencent additionally suggests disabling overlapping group offload if CPU memory is the bottleneck. ³²

But the license issue is serious enough that I would treat Hunyuan as **an evaluation/research alternative rather than your primary commercial publishing model**. The current license expressly prohibits display or use of its outputs outside the licensed Territory, which excludes the EU, UK and South Korea. ³³ Wan, by contrast, is Apache 2.0 and its maintainers say they claim no rights over generated content. ³⁴

ComfyUI and automation. On Linux, use a manually installed and pinned ComfyUI. As of August 2026, ComfyUI's official docs still say Linux is source/manual installation rather than an official Desktop package; the docs currently recommend Python 3.13, with 3.12 as a compatibility fallback, and current NVIDIA installation guidance points to PyTorch with CUDA 13.0. ³⁵

For a stable production machine:

```
git clone https://github.com/comfyanonymous/ComfyUI.git
cd ComfyUI

# After validating a known-good build:
git checkout <PINNED_COMMIT>

python3.13 -m venv .venv
source .venv/bin/activate

pip install torch torchvision torchaudio \
  --extra-index-url https://download.pytorch.org/whl/cu130

pip install -r requirements.txt

python main.py --disable-auto-launch
```

That PyTorch command reflects ComfyUI's current official NVIDIA instructions; record the actual working versions after installation rather than assuming today's latest packages will remain the production standard six months from now. ³⁵

You can then stop treating ComfyUI as exclusively graphical. It runs a local HTTP server by default and officially supports programmatic workflow submission and monitoring. ³⁶ Current Comfy CLI also supports:

```
comfy run --workflow 08_workflows/wan22_i2v_api.json --wait \
| comfy download
```

or asynchronous queueing:

```
RES=$(comfy --json run --workflow 08_workflows/wan22_i2v_api.json)
PROMPT_ID=$(echo "$RES" | jq -r .data.prompt_id)

comfy --json jobs watch "$PROMPT_ID" | comfy download
```

Those patterns are directly documented by ComfyUI. ³⁷

The automation target should therefore be:

```
edit_plan.csv
  ↓
Python
  ↓
patch prompt / seed / input image in workflow JSON
  ↓
ComfyUI API
  ↓
candidate files
  ↓
human approval
```

—not “an agent randomly clicks ComfyUI for me.”

Media normalization. `ffmpeg` is mostly about eliminating surprises, not trying to beautify already-good footage.

First inspect:

```
ffprobe -v error \
-show_entries
stream=index,codec_name,width,height,pix_fmt,r_frame_rate,avg_frame_rate \
```

```
-of json \  
06_video/approved/shot_007.mp4
```

For a 24 fps Wan source, a conservative delivery transcode is:

```
ffmpeg -y \  
-i 06_video/approved/shot_007.mp4 \  
-vf "scale=1080:1920:force_original_aspect_ratio=increase,\  
crop=1080:1920,format=yuv420p" \  
-c:v libx264 \  
-preset medium \  
-crf 18 \  
-movflags +faststart \  
-an \  
06_video/normalized/shot_007.mp4
```

I would **not** add `fps=30` merely because Shorts are vertical. Wan's native 24 fps is legitimate output, and current social guidance supports broad video-frame-rate ranges; arbitrary conversion does not invent true motion. ³⁸ If you ultimately decide on a 30 fps CapCut project, let the editor conform clips or use deliberate optical/interpolation processing only when cadence is visibly objectionable.

For still-only shots, keep lossless PNGs and let CapCut perform the pan, crop or zoom; there is no value in pre-rendering those into MP4 unless you are trying to make the CapCut assembly nearly mechanical.

Manifests, captions, and reproducibility

The most important conceptual improvement to the workflow is giving each CSV a **different authority level**.

`shot_beats.csv` or `suggested_shots.csv` is an algorithmic recommendation:

```
shot_id,start,end,duration,narration,reason,confidence  
shot_001,0.042,1.604,1.562,"Nobody expected this to work.", "opening clause",0.94  
shot_002,1.920,4.610,2.690,"In 1923, two engineers began  
experimenting.", "semantic clause",0.91  
shot_003,4.610,7.420,2.810,"with a completely different  
material.", "continuation",0.88
```

`timeline.csv` is the **human-approved timeline**:

```
shot_id,start,end,duration,narration,caption_first,caption_last,locked  
shot_001,0.042,0.790,0.748,"Nobody expected",1,1,true  
shot_002,0.790,1.604,0.814,"this to work.",2,2,true
```



```
shot_003,1.920,4.610,2.690,"In 1923, two engineers began experimenting.",
3,5,true
shot_004,4.610,7.420,2.810,"with a completely different material.",6,8,true
```

And `edit_plan.csv` tells generation/editing systems what to do with that timeline:

```
shot_id,asset_type,asset_path,visual_function,image_prompt,motion_prompt,seed,workflow,editor_not
shot_001,text_graphic,,hook,"","","","capcut","Large hook typography; hard cut"
shot_002,ai_i2v,video/shot_002.mp4,reveal,"close macro shot...","slow controlled
push-in",348219,"wan22_i2v_v4","Use first clean 0.9 sec"
shot_003,ai_still,images/shot_003.png,context,"1923 industrial
laboratory...","","927713","flux2_klein_v2","Slow 100-110% scale"
shot_004,stock,video/shot_004.mp4,explanation,"","","","","Trim to narration
boundary"
```

This distinction prevents an extremely common automation failure: the storyboard model decides that a shot would “feel better” at 4.2 seconds, while the captioning tool says it ends at 3.8 seconds and the video generator creates five seconds of footage. **Only** `timeline.csv` **gets to define edit time.**

For maximum manual control, I would retain the idea from our previous discussion of putting human break markers into a semantic transcript:

```
Nobody expected | this to work.

In 1923, | two engineers began experimenting |
with a completely different material.

But then | something impossible happened.
```

Then alignment translates those human decisions into timestamps. You think in phrases; the machine resolves the phrase boundaries to milliseconds.

The production record should also include:

```
# models.csv
role,name,revision,license,sha256
tts,chatterbox-turbo,<revision>,MIT,<hash>
image,FLUX.2-klein-4B,<revision>,Apache-2.0,<hash>
video,Wan2.2-TI2V-5B,<revision>,Apache-2.0,<hash>
```

and:

```
# generation_log.csv
asset,shot_id,model,workflow_hash,seed,prompt_hash,input_hash,created_at,approved
shot_007_take_01.mp4,shot_007,Wan2.2-TI2V-5B,...,
348219,...,2026-08-25T18:11:33Z,false
shot_007_take_02.mp4,shot_007,Wan2.2-TI2V-5B,...,
881730,...,2026-08-25T18:20:51Z,true
```

For each approved ComfyUI render, preserve:

```
model name + model SHA256
ComfyUI commit
custom-node commits
API workflow JSON
input image hash
positive prompt
negative prompt
seed
dimensions
frame count
sampler/scheduler/steps
output hash
```

ComfyUI's explicit workflow JSON formats, server APIs and CLI workflow execution make this substantially easier than it was in earlier generations of node-based tooling. ⁵

Your SRT should remain deliberately boring:

```
1
00:00:00,042 --> 00:00:00,790
Nobody expected

2
00:00:00,810 --> 00:00:01,604
this to work.

3
00:00:01,920 --> 00:00:02,580
In 1923,

4
00:00:02,600 --> 00:00:03,630
two engineers began

5
```

```
00:00:03,650 --> 00:00:04,610
experimenting.
```

Do **not** encode fonts, animation, colors or elaborate effects in the upstream file. SRT's job in this architecture is to carry text and time. CapCut Desktop parses those timecodes and turns each segment into an editable text block where styling can be applied. ⁷

Linux-to-Windows execution and CapCut assembly

Because AI generation duration varies by model, seed, cache state and chosen quality mode, a rigorous production checklist should timestamp **actual stage events**, not pretend that every project takes the same number of minutes. Wan, for example, gives an unoptimized under-nine-minute figure for its five-second 720p T12V clip while ComfyUI acceleration tests show radically different render times depending on model/LoRA configuration. ³⁹

Use `build_log.csv`:

```
event,started_at,completed_at,status,artifact
script_locked,2026-08-25T17:00:00Z,2026-08-25T17:00:00Z,pass,script_locked.md
tts_render,...,...,pass,narration_raw.wav
voice_qa,...,...,pass,narration_master.wav
alignment,...,...,pass,words.json
caption_build,...,...,pass,captions.srt
beat_review,...,...,pass,timeline.csv
storyboard_lock,...,...,pass,edit_plan.csv
asset_generation,...,...,pass,approved/
package_qa,...,...,pass,09_capcut_delivery/
```

The operational checklist is:

Timestamped gate	Action	Pass condition
<code>project_created_at</code>	Create folders, project ID, model/ environment manifest	project can be reconstructed
<code>script_locked_at</code>	Freeze narration words and pronunciation notes	no unresolved wording
<code>voice_rendered_at</code>	Generate Chatterbox takes	audio complete
<code>voice_approved_at</code>	Listen start-to-finish	no TTS glitches/ pronunciation errors
<code>master_created_at</code>	Assemble/resample to 48 kHz PCM WAV	one final narration master

Timestamped gate	Action	Pass condition
<code>alignment_completed_at</code>	Run WhisperX	every spoken word has usable alignment
<code>alignment_approved_at</code>	Spot-check first/middle/end, numbers/names	no material drift
<code>captions_generated_at</code>	Build 2-5-word SRT	no overlap/orphan mistakes
<code>beats_suggested_at</code>	Generate suggested visual beats	complete coverage
<code>timeline_locked_at</code>	Manually alter/merge/split beats	every narration moment belongs to a shot
<code>storyboard_locked_at</code>	LLM proposes visuals; human approves	every shot has a feasible asset strategy
<code>stills_approved_at</code>	Generate/select FLUX references	composition/identity locked
<code>video_approved_at</code>	Generate only needed I2V clips	acceptable motion/artifacts
<code>media_qa_at</code>	<code>ffprobe</code> , transcode bad formats	all files decode correctly
<code>package_built_at</code>	Populate CapCut folder	zero missing referenced files
<code>windows_verified_at</code>	Copy package and hash-check	Linux/Windows hashes match
<code>capcut_roughcut_at</code>	Assemble by exact timeline timecodes	visuals cover narration
<code>capcut_caption_qa_at</code>	Import/style SRT	timing/readability correct
<code>capcut_audio_qa_at</code>	Add music/SFX and audition	speech remains intelligible
<code>master_exported_at</code>	Export final vertical master	playback QA passes

Before handoff, run an automated integrity check such as:

```
find 09_capcut_delivery -type f -print0 \
| sort -z \
| xargs -0 sha256sum \
> 00_admin/checksums.sha256
```

On Windows, preserve the package rather than reorganizing it before creating the project. Keeping stable relative paths makes missing-file repair much easier.

Your CapCut delivery folder should contain **only approved or deliberately retained alternate material**, not every failed generation:

```

09_capcut_delivery/
├─ narration_master.wav
├─ captions.srt
├─ timeline.csv
├─ edit_plan.csv
├─
├─ video/
│   ├─ shot_002.mp4
│   ├─ shot_004.mp4
│   └─ shot_007_alt_b.mp4
├─ images/
│   ├─ shot_003.png
│   └─ shot_006.png
├─ audio/
│   ├─ music.wav
│   └─ sfx/
├─ provenance/
│   ├─ licenses.csv
│   ├─ generation_log.csv
│   └─ checksums.sha256

```

In CapCut Desktop, the assembly order should be:

First, establish the project. Create a 9:16 1080×1920 project and choose the project/output cadence intentionally—24 fps if your piece is overwhelmingly Wan-generated and you want to preserve that cadence, or 30 fps if mixed social/stock material makes 30 more convenient. TikTok recommends vertical 9:16 creative and safe-zone awareness rather than imposing one special “Shorts frame rate.” ²¹

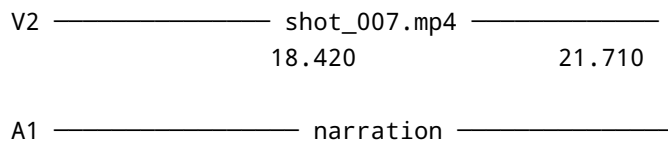
Second, put `narration_master.wav` **at timeline 00:00.000 and lock it.** From this point, the audio track is the master clock.

Third, import `captions.srt`. Current CapCut Desktop instructions are `Captions → Add Captions → Import file`; CapCut parses SRT timecodes into synchronized editable text blocks and exposes typography, position, color and animation controls. ⁷

Fourth, place visual assets according to `timeline.csv`. A record:

```
shot_007,18.420,21.710,3.290,...
```

means:



Source footage can be longer. You trim the source to the required timeline interval.

Fifth, animate static shots in the editor instead of unnecessarily pre-rendering them. A shot can be:

0.000 sec: Scale 100%, X=0
3.100 sec: Scale 112%, X=-40

That costs no generation time and can look better than poorly controlled AI motion.

Sixth, style captions. Use a reusable project template so typography is consistent across videos. Keep important words and faces away from UI-heavy edges. TikTok explicitly varies its safe-zone footprint based on interface/caption configuration, so a single universal pixel rectangle cannot guarantee safety across all posting contexts. ¹⁰

Seventh, add sound design. Keep narration dominant; use music and SFX as editorial support. For licensed music, preserve the source/license information in `licenses.csv`. On YouTube, the platform explicitly says royalty-free tracks from its Audio Library do not receive copyright claims, while Shorts longer than one minute with active copyright claims can be blocked globally. ¹¹

Eighth, inspect the full edit without stopping. Look specifically for caption/voice drift, dead visual stretches, accidental jump cuts, repeated AI compositions, morphing hands/faces, text hidden by platform UI and perceptible 24/30-fps cadence problems.

Ninth, export. A 1080×1920 H.264 master is the conservative cross-platform option. If preview quality in CapCut looks unexpectedly soft, CapCut's own support documentation notes that Proxy can reduce preview quality without indicating final-export quality; it can be disabled under performance settings for checking. CapCut also exposes custom export bitrate controls. ⁴⁰

Alternatives, cost, legal and quality tradeoffs

The ideal production philosophy is not “generate everything locally.” It is:

Run routine volume locally; buy exceptional cloud capability only when the expected value of the shot exceeds the API cost.

On a 4090, text-to-image is cheap enough that you should generate many candidate compositions. Video is where aggressive preselection matters. Wan's official baseline is under nine minutes for a five-second 720p clip without special optimization, whereas FLUX.2 Klein is specifically designed as a consumer-GPU, very-low-latency image model. ⁴¹

A sensible 45-second Short might therefore use:

Visual source	Typical role
3–5 generated I2V shots	hero/reveal/emotional motion
3–5 AI stills	locations, historical reconstructions, conceptual imagery
2–4 stock/screenshots/graphics	reality, evidence, interfaces, data
CapCut typography/animation	hooks, numbers, emphasis

That distribution is a recommendation, not a rule. The principle is that **visual change does not imply new generative video**.

The RTX 4090 itself has 24 GB of VRAM; its published total graphics power is up to 450 W. Your actual generation electricity cost should therefore be measured rather than guessed:

```
local_generation_cost =
    average_wall_power_kw
    × runtime_hours
    × electricity_price_per_kWh
```

GPU TGP alone is not the workstation's wall consumption, so use a smart plug, UPS telemetry or power meter if you care about accurate unit economics. ⁴²

The cloud comparison is interesting: Runway currently lists Gen-4.5 at \$0.12 per generated second, while BFL uses a credit system where one credit is \$0.01 and charges vary by generation model. ⁴³ Local generation therefore becomes economically attractive as output volume rises, while cloud remains attractive for difficult shots where the local model would require repeated failed attempts.

Licensing and commercial-use comparison

Component	Current issue	Production interpretation
Chatterbox	Open-source/MIT repository; generated audio watermarked with PerTh	Good local default; obtain consent for any cloned voice. ⁴⁴
FLUX.2 Klein 4B	Apache 2.0	Strong commercial local default. ³
FLUX.2 Klein 9B / dev	BFL non-commercial license	Do not casually substitute into monetized workflow. ³
Wan 2.2	Apache 2.0; project says no rights claimed over outputs	Preferred video model for broad distribution. ⁴⁵

Component	Current issue	Production interpretation
HunyuanVideo-1.5	Tencent Community License with EU/UK/South Korea exclusion applying to outputs	Serious issue for globally distributed content. ³³
Stock/music	Asset-specific licenses	Record source/license per asset; do not assume “downloadable” means reusable.
Voice cloning	Identity/consent/publicity considerations	Clone only voices for which you have appropriate permission.

AI-assisted publishing also has platform-policy consequences. YouTube currently requires disclosure where AI meaningfully creates or alters **realistic** content, while explicitly saying that script assistance, caption creation, upscaling/repair and cloning one's *own* voice do not themselves require disclosure. It also says making the appropriate AI disclosure does not by itself restrict audience or monetization eligibility. ⁴⁶

At the same time, YouTube clarified its monetization policy in July 2025 to call repetitive or mass-produced material “inauthentic content.” That makes your proposed human storyboard/editing stage strategically important: an automated factory that merely swaps nouns into the same AI template has substantially more policy risk than original scripted videos with meaningful narration and editorial work. ⁴⁷

TikTok likewise says realistic AI-generated content must be labeled and reports using creator labels, automated detection, C2PA metadata and newer watermarking techniques to identify AI-generated content. ⁴⁸

This reinforces an important production distinction:

```

GOOD AUTOMATION
timing
file naming
transcoding
asset routing
subtitle generation
workflow queueing
provenance

RISKY AUTOMATION
topic → generic script → generic AI images → generic TTS
→ same template → publish hundreds automatically

```

The former increases consistency. The latter risks reducing originality and quality.

Quality-versus-speed recommendations on your specific hardware

Choice	Faster	Higher quality/control
Chatterbox	Turbo	rerender difficult lines / use premium cloud voice
WhisperX	smaller ASR model	large-v2 + alignment QA
FLUX	Klein 4B 4-step	multiple candidates/reference-edit passes
Wan	acceleration workflow/LoRA	normal TI2V-5B final render
Hunyuan	480p I2V step-distilled 8 steps	12-step or non-distilled higher-res path
I2V	single seed	3–4 candidate seeds
Visual motion	CapCut pan/zoom	generated I2V where motion truly matters
Cloud	none	selectively escalate failed hero shots to Gen-4.5
Reproducibility	automatic prompt rewriting	fixed prompt + workflow + model hashes + seed

Hunyuan's current guidance explicitly calls eight or twelve steps the recommended balance for its distilled 480p I2V model, with four steps available at somewhat reduced quality. ⁴⁹ ComfyUI's own Wan camera-control tests similarly show that four-step acceleration is dramatically faster but may reduce movement dynamics. ²⁹

Troubleshooting and final assessment

Failure	Likely cause	Recommended fix
Captions slowly drift	audio changed after alignment; bad timestamps	confirm exact WAV hash; rerun WhisperX
One word appears wildly early/late	forced-aligner/token normalization problem	manually override that word; normalize numbers/ names; rerun
WhisperX output differs from script	ASR normalization/ hallucination	reconcile against known script instead of treating ASR as canonical
Caption contains awkward 1-word block	naïve max-word chunker	merge with neighbor unless deliberately emphasized
Shot cuts feel robotic	fixed-duration algorithm overruled semantics	manually edit <code>timeline.csv</code> ; semantic phrase beats win
TTS mispronounces a name	model doesn't know pronunciation	rewrite spelling/phonetic proxy or generate only affected chunk

Failure	Likely cause	Recommended fix
TTS voice changes between chunks	inconsistent reference/context	same reference clip/settings; provide adequate context around chunks
FLUX character changes	independent text-only generations	use approved references/multi-reference editing
I2V scene “melts”	excessive requested action	reduce motion prompt; establish strong still; request camera/subtle environmental motion
Wan 4090 OOM	incorrect model/config; other GPU processes	use TI2V-5B, documented offloading/T5 CPU flags; clear competing GPU processes
Hunyuan 4090 OOM	fragmentation/offload configuration	Tencent's <code>PYTORCH_CUDA_ALLOC_CONF</code> setting; reduce settings/disable overlap group offloading if CPU RAM is pressured. ³²
ComfyUI project works today but not later	updated nodes/models/dependencies	pin git commits, dependencies and model hashes
ComfyUI missing Wan nodes/template	outdated stable/nightly mismatch	update deliberately or load known-good workflow against known-good pinned build. ²⁸
Video imports black/oddly	unsupported pixel format/codec	transcode H.264/yuv420p with ffmpeg
Playback stutters	VFR or mixed difficult codecs	use CFR source or editor-friendly transcode; inspect with ffprobe
CapCut preview looks blurry	Proxy preview	disable Proxy for visual QA; export can still be higher quality. ⁴⁰
SRT fails to import	malformed timecodes/encoding	UTF-8, standard SRT numbering/timecodes, no overlapping malformed events. CapCut specifically recommends UTF-8 and standard timecodes. ⁷
Video generation is bottlenecking production	too many shots routed to I2V	turn low-value shots into PNG + CapCut motion/stock/graphics
Channel feels “AI-generated”	consistent synthetic visual texture	increase screenshots, data, typography, real source material and varied editorial grammar

The best pipeline in 2026 is therefore **not quite**:

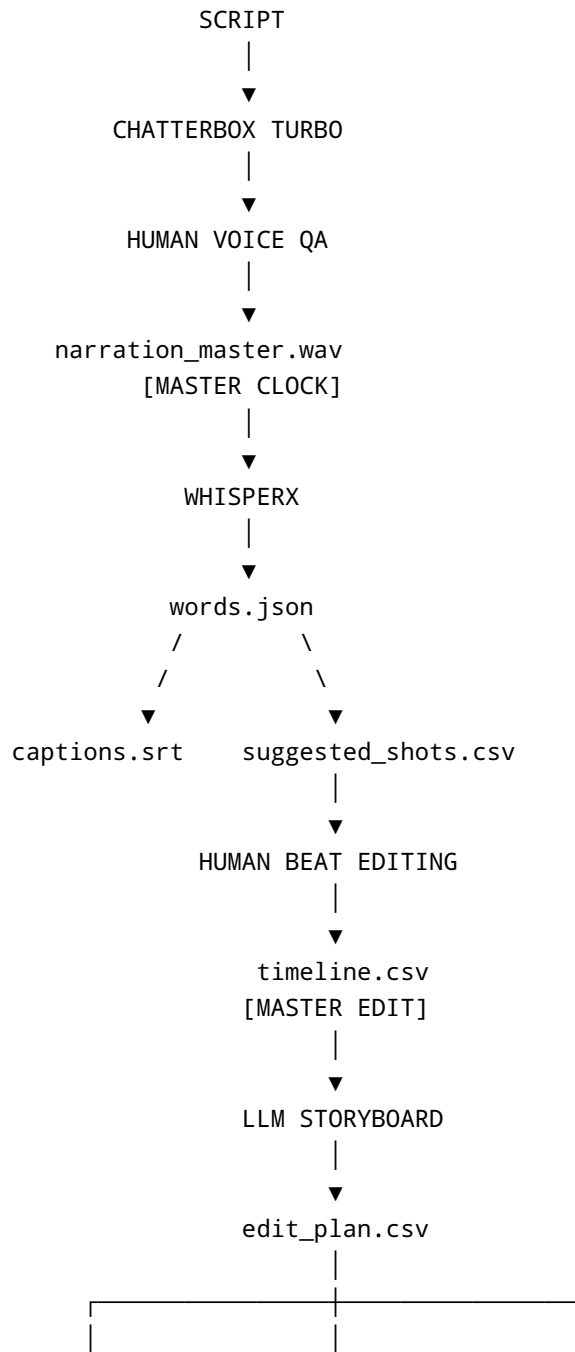
```

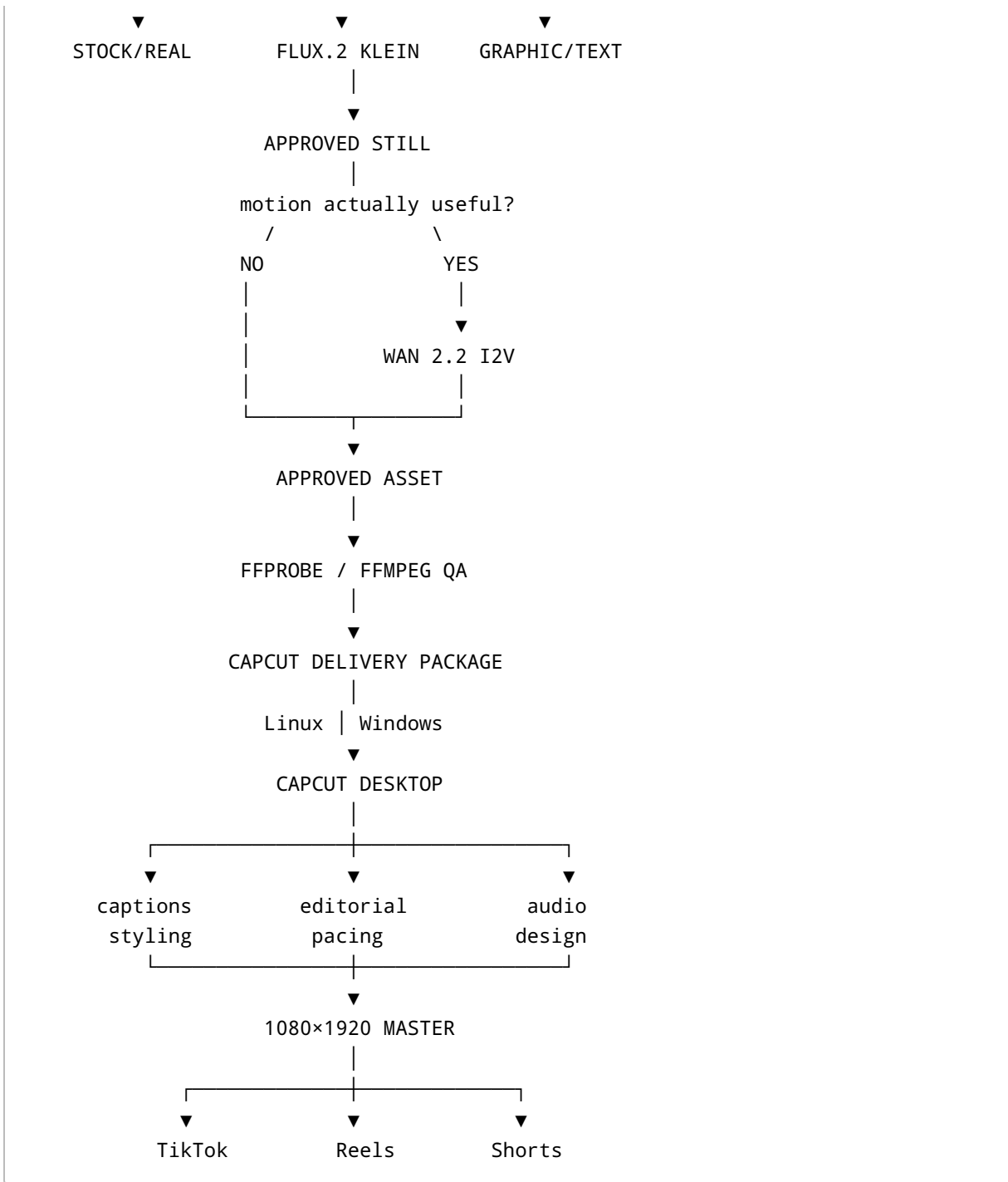
Chatterbox
→ WhisperX
→ automatic shot beats

```

- LLM
- FLUX
- Wan/Hunyuan
- ffmpeg
- CapCut

It is:





That version is strong on **cost** because expensive video generation is selective; strong on **speed** because fast still generation establishes creative direction before video rendering; strong on **quality** because humans own the two highest-leverage approval gates—voice and visual beats; and strong on **reproducibility** because prompts, seeds, model hashes, workflow JSON and media hashes can all be retained. ComfyUI's current workflow/server APIs make the generative side particularly suitable for deterministic automation. 5

It is also better legally than the original design if you default to Chatterbox + FLUX.2 Klein 4B + Wan 2.2, because those components have substantially clearer open licensing than substituting non-commercial FLUX variants or HunyuanVideo-1.5 into a globally published commercial workflow. ⁵⁰

Most importantly, the architecture puts automation in places where machines are good at it—**alignment, timing, file generation, queuing, transcoding, metadata and candidate generation**—while preserving human judgment for **pacing, visual metaphor, shot choice, typography, humor, emphasis and final rhythm**. That division is more consistent with platform guidance than trying to maximize synthetic generation for its own sake: TikTok's current guidance emphasizes a platform-native, immediately engaging vertical experience rather than maximal polish, while YouTube's monetization rules explicitly favor original/authentic work over repetitive mass-produced templates. ⁵¹

¹ ¹⁷ ¹⁸ GitHub - m-bain/whisperX: WhisperX: Automatic Speech Recognition with Word-level Timestamps (& Diarization) · GitHub

<https://github.com/m-bain/whisperx>

² ¹⁴ ¹⁵ ⁴⁴ GitHub - resemble-ai/chatterbox: SoTA open-source TTS · GitHub

<https://github.com/resemble-ai/chatterbox>

³ ²⁴ ²⁵ ⁵⁰ GitHub - black-forest-labs/flux2: Official inference repo for FLUX.2 models · GitHub

<https://github.com/black-forest-labs/flux2>

⁴ ⁶ ¹² ²⁷ ³⁴ ³⁸ ³⁹ ⁴¹ ⁴⁵ GitHub - Wan-Video/Wan2.2: Wan: Open and Advanced Large-Scale Video Generative Models · GitHub

<https://github.com/Wan-Video/Wan2.2>

⁵ ComfyUI Server Routes (HTTP and WebSocket API) - ComfyUI

https://docs.comfy.org/development/comfyui-server/comms_routes

⁷ How Do I Import Subtitles?

<https://www.capcut.com/help/how-to-import-subtitles>

⁸ ³³ HunyuanVideo-1.5/LICENSE at main · Tencent-Hunyuan/HunyuanVideo-1.5 · GitHub

<https://github.com/Tencent-Hunyuan/HunyuanVideo-1.5/blob/main/LICENSE>

⁹ ²¹ ⁵¹ Creative best practices for performance ads

<https://ads.tiktok.com/resources/help/article/creative-best-practices?lang=en>

¹⁰ TikTok Auction In-Feed Ads | TikTok Ads Manager

<https://ads.tiktok.com/help/article/tiktok-auction-in-feed-ads>

¹¹ Understand three-minute YouTube Shorts - YouTube Help

<https://support.google.com/youtube/answer/15424877?hl=en>

¹³ ³⁵ System Requirements - ComfyUI

https://docs.comfy.org/installation/system_requirements

¹⁶ Text to Speech | ElevenLabs Documentation

https://elevenlabs.io/docs/overview/capabilities/text-to-speech?utm_source=chatgpt.com

¹⁹ Wrong word-level timestamps in force-alignment starting ...

https://github.com/m-bain/whisperX/issues/1220?utm_source=chatgpt.com

20 Flux Word-Level Timestamps

https://developers.deepgram.com/changelog/2026/7/2?utm_source=chatgpt.com

22 Creating with Gen-4.5

https://help.runwayml.com/hc/en-us/articles/46974685288467-Creating-with-Gen-4-5?utm_source=chatgpt.com

23 31 32 GitHub - Tencent-Hunyuan/HunyuanVideo-1.5: HunyuanVideo-1.5: A leading lightweight video generation model · GitHub

<https://github.com/Tencent-Hunyuan/HunyuanVideo-1.5>

26 Overview

https://docs.bfl.ai/quick_start/pricing?utm_source=chatgpt.com

28 Wan2.2 Video Generation ComfyUI Official Native Workflow Example - ComfyUI

https://docs.comfy.org/tutorials/video/wan/wan2_2

29 ComfyUI Wan2.2 Fun Camera Control: Video Generation Workflow Example - ComfyUI

<https://docs.comfy.org/tutorials/video/wan/wan2-2-fun-camera>

30 API Changelog & Updates | Runway Dev

https://docs.dev.runwayml.com/api-details/api_changelog/?utm_source=chatgpt.com

36 Server Overview - ComfyUI

https://docs.comfy.org/development/comfyui-server/comms_overview

37 Getting Started - ComfyUI

<https://docs.comfy.org/comfy-cli/getting-started>

40 Why Does the Video Quality Change After Exporting?

<https://www.capcut.com/help/video-quality-change-after-exporting>

42 GeForce RTX 4090 Graphics Cards for Gaming

https://www.nvidia.com/en-us/geforce/graphics-cards/40-series/rtx-4090/?utm_source=chatgpt.com

43 Runway Developer Portal

https://dev.runwayml.com/?utm_source=chatgpt.com

46 Disclosing use of GenAI content - Android - YouTube Help

<https://support.google.com/youtube/answer/14328491?co=GENIE.Platform%3DAndroid&hl=en>

47 YouTube channel monetization policies - YouTube Help

<https://support.google.com/youtube/answer/1311392?hl=en>

48 TikTok shares more ways to spot, shape and understand AI-generated content - Newsroom | TikTok

<https://newsroom.tiktok.com/tiktok-ssa-shares-more-ways-to-spot-shape-and-understand-ai-generated-content?lang=en-ZA>

49 HunyuanVideo-1.5/assets/step_distillation_comparison.md ...

https://github.com/Tencent-Hunyuan/HunyuanVideo-1.5/blob/main/assets/step_distillation_comparison.md?utm_source=chatgpt.com